

TRANSLATED TO ENGLISH USING GOOGLE TRANSLATOR

Syllabus for Software Engineering in Applied AI

1 Fundamentals of AI and LLMs for Programmers

- **History and context of AI**
 - Where did the first idea of an "intelligent machine" in humanity come from?
 - Timeline leading up to ML, DL, LLMs, and current tools.
- **Introduction to LLMs (Large Language Models)**
 - Review of ML, DL, and AI to prepare the ground.
 - How LLMs work: transformers, incorporation, attention
 - Examples of LLMs: ChatGPT, Gemini, and others.
- **Fundamentals of Web Machine Learning**
 - Difference between Machine Learning, Deep Learning, and Artificial Intelligence
 - How tensors work and why they are the basis of models.
 - The concept of neural networks and how they learn.
- **Creating your first AI from scratch**
 - Step-by-step guide to creating and training your first neural network in JavaScript.
 - Understanding the complete cycle: data → training → validation → inference
- **Practical projects with ML on the Web**
 - Using ML with video in the browser (e.g., detection, tracking, filters, etc.)
 - Applying ML in games (e.g., intelligent bots, assistants, dynamic effects)
 - A practical introduction to Deep Learning in real-world scenarios.

- **Prompt Engineering in practice**
 - How to write prompts that actually generate good responses.
 - Prompt patterns for coding, debugging, documenting, and learning faster.
- **AI tools to accelerate your life as a developer.**
 - History and overview of Cursor
 - Comparing VSCode, Windsurf, and Cursor
 - How to use `.cursor` rules for customizing AI "to your liking"
 - Vibe Coding concept and the Levelsio case study (microSaaS with over 100k/month revenue using AI)
- **MCPs and automation for developers**
 - What are MCPs (Model Context Protocols) and how do they connect AI with APIs/services?
 - Using AI for:
 - Generate automated tests
 - Consult updated documentation.
 - Browsing websites and extracting information.
 - Tools like N8N MCP and similar ones for orchestrating flows with AI.
- **RAG, embeddings, and semantic search**
 - What is RAG (Retrieval-Augmented Generation) and why is it so important?
 - What are embeddings and Vector Databases?
 - The concept of similarity search (searching for "meaning," not just exact text).
 - Project: Creating the first RAG with JavaScript and Postgres

- **Open-source vs. proprietary models**
 - Advantages and disadvantages of open and closed models.
 - How to use OpenRouter to orchestrate multiple models
 - How to run models locally with Ollama
- **Agents and advanced automation**
 - What are AI agents and how do they make decisions in stages?
 - Real-world examples of autonomous agents that write code and accelerate developer work.

2 - Generative AI and Prompt Engineering APIs

- **Overview of the AI-as-a-Service market**
 - Cases of companies that are "ChatGPT wrappers" and have raised millions of dollars.
 - Understanding where the real opportunities lie for developers and businesses.
- **Leading AI model providers**
 - Overview of OpenAI, Anthropic, Hugging Face, and Google Gemini
 - Differences in APIs, models, and use cases.
- **Advanced prompt engineering**
 - Prompt chaining (chaining prompts for complex tasks)
 - Prompt templates to standardize workflows and reuse prompts.
 - Strategies to reduce hallucinations and improve accuracy.
- **Good practices for consistency and cost-efficiency.**
 - How to design API calls with cost in mind.
 - Techniques for caching, context reuse, and token reduction.

- Patterns to make model behavior more predictable
- **Advanced RAG in practice**
 - Using LangChain, Pipes (or similar) to orchestrate AI flows.
 - Error and retry management strategies
 - Best practices for logging, observability, and debugging in AI apps.
- **Hands-on: Integrating AI into the back-end**
 - Project: Integrate an OpenAI LLM with an existing back-end.
 - Request flow, authentication, versioning, and testing.
- **Multimodal models (text, image, audio, video)**
 - When to use text, image, audio, and video templates.
 - Overview of OpenAI Vision, Whisper, Gemini multimodal and others.
- **Practical applications with a focus on APIs.**
 - Building a smart OCR to extract and understand documents.
 - Media analysis (image/video) for insights and automation.
 - Creating multimodal bots that understand text, images, audio, and video together.

3 - MCP – Model Context Protocol

- **Overview of MCP (Model Context Protocol)**
 - What is MCP and why is it gaining so much traction now?
 - How does it resolve the chaos of custom integrations between LLMs and services?
- **Standardized integration of LLMs with MCP**
 - How to use MCP to connect LLMs to APIs, databases, and internal services.

- Standardization of input/output and context across different models and tools.
- **MCPs vs. Traditional Tools**
 - Difference between MCPs and the classic "tools/plugins" model
 - When does it make sense to use MCP instead of each provider's proprietary tools?
- **MCP em JavaScript**
 - How to create and consume MCPs using JavaScript/TypeScript
 - Good practices for code organization and project structure.
- **Transforming your company into an "MCP"**
 - Expose internal services (CRM, billing, support, product data) via MCP.
 - Design an AI-ready layer for the entire company's technology stack.
- **Security and governance in MCPs**
 - Authentication and Service Tokens
 - Rate limiting to prevent abuse and unexpected costs.
 - Web Application Firewall (WAF) and protections to securely expose MCPs to LLMs and agents.
- **Real-world use cases in companies**
 - How companies use MCPs to connect AI to legacy systems.
 - Examples of automation, internal copilots, and corporate assistants.
- **Practical application scenarios**
 - Plugins and smart agents consuming MCPs
 - Integrations with IDEs (custom copilots)
 - Connections to corporate systems (ERP, CRM, support, analytics)

- **Hands-on: From zero to MCP in production**
 - Creating an MCP server from scratch
 - Integration of this MCP with front-end (UI) and back-end.
 - Testing the MCP on different LLM clients and combining it with other MCPs.

4 - Creation of Autonomous Agents

Unit 1 — Agent Architecture

- Concept of *agent loop*: Perception → reasoning → action → feedback.
- Internal structure: *planner, executor, memory store, toolbox*.
- Types of agents: *task-based, interactive, goal-oriented, autonomous*.
- Componentization: prompts, tools, context, and feedback loops.

Unit 2 — Patterns of Reasoning and Execution

- **ReAct Pattern**: reasoning + iterative action (reasoning traces).
- **Plan-and-Execute Pattern**: Decomposition of objectives and structured execution.
- **Reflection Pattern**: Self-assessment and behavior correction.
- A practical comparison between ReAct and Plan-Execute in different scenarios.
- Implementations using LangChain, LangGraph, and equivalent frameworks.

Unidade 3 — Function Calling e Tool Use

- Principles of *tool use* and *function calling* in LLMs.
- Design of *schemas* JSON for executable functions.
- Connecting agents to REST APIs, databases, automations, and MCPs.
- Practical implementation: an agent that creates, executes, and adjusts tasks using external tools.

Unit 4 — Memory and Reflection in Autonomous Agents

- Types of memory: short-term, long-term, episodic, and contextual.
- Storage and retrieval strategies via embeddings.
- *Reflection loops* How to allow agents to review and improve their own logic.
- Use cases: personal assistants, automated debugging, and incremental learning.

Unit 5 — Context Management

- What is an "active context" and how to structure it?
- Techniques of *context pruning* and *context stitching*.
- Shared contexts among multiple agents.
- Balancing global context (collective memory) and local context (individual memory).

Unit 6 — LangGraph and Complex Workflows

- Introduction to **LangGraph** as an orchestrator of multi-agent flows.
- Creating execution graphs and controlling dependencies.
- Task routing, fallback handlers, and retry logic.
- Monitoring and debugging connected agents in graphs.

Unit 7 — Observability and Limits of Autonomy

- Metrics and logs of agent execution.
- Auditing and traceability of autonomous decisions.
- Implementation of guardrails and *human-in-the-loop*.
- Scope definition and ethics in the autonomy of agents.
- Prevention of *runaway loops* and API abuse.

Unit 8 — Practical Project: Agent that Automates Complex Tasks

- Development of a **full autonomous agent**, with:
 - adaptive planning;
 - Executing tasks via external APIs;
 - Log recording and feedback loops;
 - context control and persistent memory.
- Integration with **LangGraph** to orchestrate subtasks.
- Presentation and evaluation based on metrics of effectiveness, autonomy, and resilience.

Unit 9 — Multi-Agent Systems

- Concepts of collaboration between agents: *orchestration, coordination and negotiation*.
- Standards: *Supervisor, Hierarchical, Group Chat, Delegation and Consensus*.
- Asynchronous communication and message exchange.
- Implementation of a **cooperative multiagent mini system** (e.g., analyst → planner → executor).
- Real-world examples: Claude Team, AutoGPTs, CrewAI, Agentarium (practical reference).

5 AI tools for UX & UI

Unit 1: The AI Revolution in Interface Design and Development

- Introduction to the concept of AI-driven UX/UI.
- Overview of AI tools for each stage of the process: research, ideation, prototyping, development, and testing.
- Use case analysis: how AI is accelerating the creation of digital products.

Unit 2: Prototyping and UI Generation with Artificial Intelligence

- Generating wireframes and mockups from natural language descriptions (Text-to-UI).
- AI-assisted visual prototyping tools.

- Hands-on: Using Firebase Studio to transform Figma designs into front-end code and analyze the results.
- Validation of usability and user flows with the aid of AI models.

Unit 3: AI Agents and CLI in the Front-End Developer Workflow

- What are encoding agents and how do they operate in a development environment?
- Using the Gemini CLI for project scaffolding, component generation, and code refactoring.
- Defining a workflow with AI agents: from idea to functional and testable component.
- Best practices for prompt engineering focused on code generation (HTML, CSS, JavaScript/TypeScript).

Unit 4: Automation and Intelligent Interaction with the UI

- End-to-End (E2E) test automation with autonomous agents that consume the application's MCP.
- Practical Discussion: Using a tool like mcpui.dev to visualize and interact with the context exposed by the application via MCP, facilitating debugging and agent development.

Unit 5: Integrating AI Logic into the Client and Server

- Consuming AI APIs directly from the front-end: use cases and security challenges.
- Using Firebase AI Logic to create, manage, and deploy custom backends for front-end applications with embedded AI logic.
- Developing intelligent features: semantic search, support chatbots, and real-time content personalization.
- Hands-on: Building an application that uses AI logic hosted on Firebase to process user input.

6 AI tools for DevOps

UNIT I - Fundamentals of Generative AI for Infrastructure

- 1.1. Large Language Models (LLMs) Architecture
- 1.2. APIs de IA: OpenAI, Anthropic Claude, AWS Bedrock
- 1.3. Autonomous agent frameworks: LangChain, CrewAI, AutoGen
- 1.4. RAG (Retrieval-Augmented Generation) applied to technical documentation.
- 1.5. Prompting patterns for infrastructure tasks
- 1.6. Limitations, hallucinations, and validation strategies

UNIT II - IaC Copilot: Intelligent Infrastructure Generation

- 2.1. Natural language translation to Infrastructure as Code
- 2.2. Automated generation of Terraform, Pulumi, and Helm Charts
- 2.3. Automated validation: Terraform Plan, Terraform Validate
- 2.4. Policy-as-Code com OPA (Open Policy Agent) e Sentinel
- 2.5. Workflow PR-first with AI-assisted review
- 2.6. Infrastructure drift detection and correction suggestions.

Practice 1:Development of a copilot for generating validated Terraform modules.

UNIT III - Agents for Kubernetes: Deployment and Operation

- 3.1. Generation and tuning of YAML manifests via natural language processing.
- 3.2. Intelligent configuration of HorizontalPodAutoscaler (HPA) and VerticalPodAutoscaler (VPA)
- 3.3. Autoscaling based on workload and custom metrics
- 3.4. Automated management of Ingress, Services, and Network Policies.
- 3.5. AI-driven rollout/rollback strategies (canary, blue-green, progressive delivery)
- 3.6. Integration with GitOps: Argo CD and Flux

Practice 2:Agent for end-to-end deployment with readiness validation.

UNIT IV - Assisted Troubleshooting and Intelligent Diagnosis

- 4.1. Framework ReAct (Reasoning + Acting) para troubleshooting estruturado
- 4.2. Automated diagnosis of common Kubernetes errors - CrashLoopBackOff, ImagePullBackOff, OOMKilled
- 4.3. Intelligent analysis of distributed logs and stack traces
- 4.4. Correlation of traces (Jaeger, Zipkin) and metrics for root cause analysis.
- 4.5. Detection of latency, 5xx errors, and performance degradation.
- 4.6. Automated suggestion of fixes and remediations

Practice 3:Automatic diagnostic system for pod problems.

UNIT V - AIOps and Intelligent Observability

- 5.1. Generating PromQL and LogQL queries from natural language.
- 5.2. Automated creation of contextualized Grafana dashboards
- 5.3. Anomaly detection with Machine Learning (Prophet, Isolation Forest)
- 5.4. Intelligent correlation between metrics, logs, and traces.
- 5.5. Predictive alerts and reduction of false positives.
- 5.6. Trend analysis and resource forecasting

Practice 4:Intelligent dashboard with real-time anomaly detection.

UNIT VI - ChatOps with Human Approval and Governance

- 6.1. Conversational bot architecture for Slack, Discord, and Microsoft Teams
- 6.2. Implementation of secure commands: /deploy, /scale, /rollback, /investigate
- 6.3. Visualization of differences and impact analysis of changes.
- 6.4. Approval workflows with guardrails and circuit breakers.
- 6.5. Integration with RBAC (Role-Based Access Control)
- 6.6. Auditing and traceability of actions performed.

Practice 5: ChatOps bot with approval workflow for deployments.

UNIT VII - AI-Assisted Security and Compliance

- 7.1. Automated scanning of IaC and containers (Snyk, Trivy, Checkov)
- 7.2. Automatic "AI fix" for detected vulnerabilities
- 7.3. Detection and prevention of classified leaks.
- 7.4. Explanation of takeover bid policies in natural language.
- 7.5. Guided remediation of CVEs and critical vulnerabilities.
- 7.6. Generation of compliance reports (SOC2, ISO 27001, PCI-DSS)

UNIT VIII - CI/CD Copilot and Intelligent Pipelines

- 8.1. Automated pipeline generation (GitLab CI, GitHub Actions, Jenkins)
- 8.2. Smart gates: automated testing, SAST, DAST, dependency scanning
- 8.3. Impact analysis of changes in Pull Requests
- 8.4. Recommendations for deployment strategies (canary, blue-green)
- 8.5. Build time optimization and smart caching
- 8.6. Automatic rollback based on quality metrics.

UNIT IX - FinOps and Cost Optimization with AI

- 9.1. Cost estimation for Pull Requests with Infracost
- 9.2. Rightsizing of pods and nodes based on usage history.
- 9.3. Recommendations for spot and reserved instances
- 9.4. Detection of idle resources and "zombie resources"
- 9.5. Cost forecasting with Machine Learning
- 9.6. Storage and networking optimization

UNIT X - Runbook and Automatic Post-mortem RAG

- 10.1. RAC architecture for internal technical documentation.
- 10.2. Indexing of playbooks, READMEs, and corporate wikis.
- 10.3. An agent that automatically queries and executes runbooks.
- 10.4. Automatic generation of incident timelines

- 10.5. Extracting lessons learned
- 10.6. Automated blameless post-mortem templates

UNIT XI - Safe Self-Remediation with Guardrails

- 11.1. Playbooks triggered automatically by alerts
- 11.2. Flow: alert → verification → mitigation → validation
- 11.3. Automatic canary with intelligent rollback.
- 11.4. Circuit breakers and rate limiting in automated trading.
- 11.5. Simulation (dry-run) before production execution.
- 11.6. Human-in-the-loop for critical and high-impact actions.

UNIT XII - Integrative Project and Practical Cases

- 12.1. Full agent or copilot development
- 12.2. Integration of multiple components (IaC + K8s + Observability)
- 12.3. Implementation of guardrails and safety policies.
- 12.4. Validation tests and simulation of real-world scenarios.
- 12.5. Presentation of projects and discussion of results.
- 12.6. Feasibility analysis, ROI, and best practices

7 AI Tools for Project Management

Unit 1: Planning and Scope with AI (Requirements Copilot)

- Apply AI tools to support the gathering and definition of software requirements;
- Convert natural language (NL) descriptions into epics, user stories, and change requirements;
- Perform task decomposition and define acceptance criteria using generative models.

Unit 2: Intelligent Backlog Prioritization

- Utilize prioritization techniques and frameworks (RICE, WSJF, MoSCoW) with AI support;
- Simulate the impact of features and generate automatic roadmap suggestions;
- Apply smart scoring criteria for backlog refinement.

Unit 3: Schedule, Capacity, and Assisted Allocation

- Using AI for dynamic schedule creation and identification of dependencies between tasks;
- Analyze "what-if" scenarios and perform team capacity balancing;
- Automate schedule adjustments based on load forecasts and constraints.
operational.

Unit 4: Estimates and Forecasts

- Apply AI-based techniques for forecasting deadlines, costs, and effort;
- Analyze historical data to improve the accuracy of estimates;
- Employ probabilistic simulations (e.g., Monte Carlo) for uncertainty analysis.

Unit 5: Risks and Mitigations with Project AIOps

- Utilize AIOps techniques for automatic detection of scope, schedule, and resource risks;
- Analyze patterns and anomalies in project data to predict failures and bottlenecks;
- Generate automated mitigation plans based on historical data and heuristics.

Unit 6: Turbocharged Meetings (Notes, AIs, Follow-ups)

- Apply AI models to transcribe and summarize team meetings;
- Identify decisions, pending issues, and actions based on conversations in natural language;
- Integrate AI solutions into project calendars and boards.

Unit 7: Status Reports and Executive Summaries

- Automate the generation of sprint, project, and portfolio reports with AI;
- Adapt communications for different audiences (technical, managerial, board of directors);
- Consolidate performance and productivity data into reports in natural language.

Unit 8: Governance, Compliance and Quality

- Applying AI for compliance verification and traceability in projects;
- Create automated checklists and decision trails for internal audits;
- Automate the generation and control of quality documentation;
- Implement automated checks in JavaScript for auditing and version control.

Unit 9: Automation in Jira/Asana/Trello/Notion/Slack

- Create intelligent automations integrated with management and collaboration platforms;
- Develop JavaScript bots for creating, updating, and prioritizing cards and tasks;
- Use natural language (NL → workflow) for generating and approving tasks;
- Apply automated business rules for reminders, notifications, and approvals.

Unit 10: Portfolio and OKRs with AI

- Applying AI to align projects with objectives and key results (OKRs);
- Analyze outcome metrics and prioritize initiatives in corporate portfolios;
- Evaluate the impact of projects and propose adjustments based on data.

8- Systems Architecture with AI

Unit 1: Fundamentals of AI-First Architecture

- Understanding the differences between traditional architectures and AI-driven architectures;
- Analyze the five main design patterns in intelligent systems: *Prompting, Responsible AI, UX, AI-Ops* and *Optimization*;
- Apply a *decision framework* to identify when to use AI versus traditional deterministic rules;
- Discussing trade-offs between latency, accuracy, cost, and performance.

Unit 2: Single-Agent Architectures

- Understanding the concept of autonomous agents and their role in linear decision flows;
- Analyze agent patterns: *Reactive, Memory-Enhanced, Tool-Using, ReAct* and *Self-Reflection*;
- Identify ideal application contexts (short tasks, limited budgets, low latency);
- Develop prototypes of simple agents in JavaScript integrating generative AI APIs;
- Simulate interactions between the agent and users in real-world software environments.

Unit 3: Multi-Agent Architectures

- Understanding the role of collaboration between agents and its importance in complex systems;
- Exploring multi-agent orchestration patterns: *Sequential, Parallel, Supervisor, Hierarchical, Group Chat* and *Handoff*;

- Evaluate distributed challenges such as node failures, coordination, state control, and synchronization;
- Implement, in JavaScript, examples of communication between multiple agents using message queues and asynchronous calls;
- Discuss use cases of multi-agent interactions in corporate environments (e.g., analytics pipelines, collaborative chatbots, expert agents).

Unit 4: AI-Specific Design Patterns

- Apply standards *Retrieval-Augmented Generation (RAG): Basic, Hybrid Search, Multi-Index* and *Agentic RAG*;
- Utilize intelligent routing strategies: *Model Router* and *Intent-Based Routing* for cost and latency optimization;
- Implement mechanisms of *caching* and response reuse (Semantic Cache, Prompt Cache, Response Streaming);
- Incorporate mechanisms of *Human-in-the-Loop (HITL)* as *Approval Gates*, *Confidence Thresholds* and *Audit Trails*.

Unit 5: Enterprise Architecture

- Designing complete AI architectures for production, focusing on modularity, scalability, and security;
- Map the *stack* Complete: API Gateway → Orchestration (Kubernetes) → Shared Services → Observability;
- Adopting design principles such as *Loose Coupling*, *Clear Interfaces* and *Policy-Driven Control*;
- Employing practices of *observability in AI*, including tracking prompts, logs, and performance metrics;
- Discussing deployment models in hybrid environments (Kubernetes, Serverless, and Edge);
- Implementation of examples of model routing simulation and cost control (*Model Tiering*).

9 - Data Processing and Model Fine-Tuning

Unit 1: When to do fine-tuning (decision framework)

- Designing complete AI architectures for production, focusing on modularity, scalability, and security;
- Map the *stack* Complete: API Gateway → Orchestration (Kubernetes) → Shared Services → Observability;
- Adopting design principles such as *Loose Coupling*, *Clear Interfaces* and *Policy-Driven Control*;

Unit 2: Preparing Datasets for Fine-Tuning

- Identify and collect relevant data for training customized models;
- Apply basic techniques for cleaning, balancing, and standardizing textual data;
- Structuring datasets in JSONL format, according to fine-tuning API specifications;
- Automate data preparation with JavaScript scripts using text manipulation libraries;
- To assess the importance of data quality and diversity in the final result of the model.

Unit 3: Fine-tuning via OpenAI API (upload and train)

- Understanding the fine-tuning process in commercial APIs (e.g., OpenAI, Gemini).
- Perform the dataset upload and monitor the model training step by step;
- Configure basic hyperparameters and monitor training progress.
- Implement JavaScript scripts to automate the upload process and track fine-tuning;
- Discuss best practices for versioning and documenting fine-tuned models.

Unit 4: LoRA and PEFT (practical concepts)

- Understanding the principles of *Low-Rank Adaptation (LoRA)* and *Parameter-Efficient Fine-Tuning (PEFT)*;
- Apply subtle fine-tuning strategies to reduce costs and accelerate training;
- Evaluate trade-offs between performance and parameterization efficiency;
- Implement practical demonstrations in JavaScript using APIs that support adaptive layers;
- Compare the behavior of fully fine-tuned models versus LoRA in specific tasks.

Unit 5: Evaluating fine-tuned models

- Apply qualitative and quantitative evaluation metrics to adjusted models;
- To measure the accuracy, consistency, and appropriateness of responses in different scenarios;
- Use A/B testing to compare custom and generic models;
- Implement JavaScript scripts for automating performance testing and logging results;
- Discussing risks of *overfitting* and loss of generalization in fitted models.

Unit 6: Final Project: Customized Model for a Specific Domain

- To plan and develop a fine-tuned model focused on a specific domain (e.g., customer service, technical support, legal, financial);



- Apply the concepts learned regarding data preparation, adjustment, and evaluation;
- Integrate the customized model into a practical workflow, such as a chatbot or automated agent;
- Document and present the results, justifying the architectural and approach choices;
- Implement a functional prototype in JavaScript that uses the model trained via API.

10 - Security and Governance in AI

- What is Governance in AI?
- Interpretability and Explainability in AI Models
- Biases and Responsibility in AI Models
- Risk Management in Artificial Intelligence - Human and Ethical Aspects
- Risk Management in Artificial Intelligence - Security and Data
- Risk Management in Artificial Intelligence - Legal and Regulatory Aspects
- Financial Costs in Artificial Intelligence

11 - Integrated Project – Capstone Project

Unit 1: Ideation and Architecture of Micro-SaaS

- Defining the problem and scope of Micro-SaaS.
- Feasibility analysis: identifying the target audience and building the value proposition.
- Solution architecture design, encompassing front-end, back-end, and AI components (RAG, Agents, Orchestration).
- Project planning: defining milestones, initial backlog, and technologies (stack).

Unit 2: Fundamentals of Solution - RAG and Agents

- Building the core of intelligence: implementing a Retrieval-Augmented Generation (RAG) system.
- Selection and configuration of Vector Databases
- Development of the main agent that will use RAG to respond to complex queries.
- Delivery 1: Functional prototype of the RAG system with a command-line interface (CLI) for logic validation.

Unit 3: Orchestration, Back-end, and the Intelligent Interface

- Developing back-end integration: creating APIs.
- Implementing the IA-UI Bridge: Enabling the Model-Context Protocol (MCP) in the application to expose the service's capabilities in a standardized way to agents.
- Orchestration of multiple agents or LLMs using the ADK (Agent Development Kit).

- Delivery 2: Documented and functional back-end API, and the front-end application with MCP enabled, ready for programmatic interaction.

Unit 4: Front-End Development, Interaction, and Deployment

- Development of the Micro-SaaS user interface (UI) in Angular, already structured with MCP.
- Front-end integration with back-end APIs to consume AI services.
- MCP Validation: Using tools to validate that agents can correctly interpret and interact with the front-end through the protocol.
- CI/CD pipeline configuration and solution deployment.
- Delivery 3: Complete solution deployed, with the main UI functional and the ability to demonstrate an agent's interaction with the front-end via MCP.

Unit 5: Final Presentation and Technical Defense

- Structuring the project presentation: storytelling, live demonstration, and results achieved.
- Preparation for technical defense: ability to justify architectural, technological, and implementation decisions.
- Finalization of project documentation: Clear README, architecture diagram, and instructions for use.
- Final evaluation based on complexity, functionality, code quality, presentation, and defense of the project.

12 - Career and Interviews for Applied AI Engineers

- **LinkedIn Optimization for Recruiters** AI keywords, high SSI, impactful headline, "About" section with projects.
- **Strategic Networking** Connect with tech recruiters and leads, participate in AI communities, and maintain consistent engagement.
- **Personal Branding and Positioning** - Share learnings, technical articles, projects, and position yourself as an expert in applied AI.
- **Professional GitHub Portfolio** - Deployed projects, detailed READMEs, clean code, accessible live demos
- **Expectations by Level (Junior → Senior)** - What each level requires: scope, autonomy, impact, technical leadership
- **Preparation for HR Interviews** - STAR methodology, effective storytelling, cultural alignment, addressing weaknesses strategically
- **Salary Expectations and Value Proposition** Market research: how to justify its value, and when to mention numbers.
- **Live Coding e Problem Solving** - Practice coding interviews, think-aloud, edge case testing, optimization

- **System Design for AI Systems-** Architecting complete systems, justifying trade-offs, scalability, costs, resilience.
- **Explaining Technical Decisions-** Communicate architectural choices, explaining why a particular design was chosen, and the alternatives considered.
- **Specific AI Technical Questions**RAG vs. fine-tuning, when to use agents, cost optimization, failure handling
- **Effective Salary Negotiation**Right timing, salary anchors, multiple offers as leverage, complete package beyond salary.
- **Request for Raise/Promotion-** Documenting impact, strategic timing, building a case with metrics, professional presentation.
- **Differentiation Techniques-** Demonstrate live projects, case studies of results, open source contributions, and technical expertise.